DDX3216 Remote

Software System Manual

Version 2.0 — .NET 10

Repository: DDX3216GUI · Main branch · Commit ec42856 plus the uncommitted .NET 10 upgrade and dead-code removal

Prepared for technical teams, support teams, administrators, and business stakeholders

Document date: 12 October 2026

Professional delivery edition

Document Control

Revision History

Version	Date	Author	Description
1.0	12 October 2026	Nelson Fernandes	Updated customer-ready system manual based on version 2.0 source documentation and the .NET 10 upgrade.

Document Conventions

- **Security consideration** identifies controls, exposures, or required safeguards.
- **Best practice** identifies a recommended operational or engineering approach.
- **Assumption** identifies behavior inferred from code analysis and requiring validation against the physical mixer.

Scope note: This manual documents DDX3216 Remote version 2.0. Statements concerning physical-console behavior that were not confirmed by source evidence are explicitly marked as assumptions.

Table of Contents

Document Control 2

Revision History 2

Document Conventions 2

Table of Contents 2

1. Executive Summary 6

Recommendations 6

2. Business Overview 6

Business Objectives 6

Stakeholders 7

Scope and Limitations 7

3. System Architecture 7

Logical Architecture 7

Architecture Diagram 7

Primary Components 8

Change Data Flow 8

4. Functional Overview 9

Core Capabilities 9
Role Matrix 9
Business Rules 9
5. Technical Architecture 10
Technology Stack 10
Frontend Design 10
Backend Design 10
SysEx Protocol 11
.NET 10 Upgrade Validation 11
6. Database Design 11
Entity Summary 11
Relationship Diagram 12
Data Protection and Backup 12
7. Authentication and Security 12
Authentication Model 12
Session and Authorization Flow 12
Security Controls and Considerations 13
8. API Reference 13
Conventions 13
HTTP Status Codes 13
Endpoint Catalog 14
Operational Endpoints 15
WebSocket Contract 15
9. Configuration Guide 16
Application Settings 16
Environment 17
Fixed Values 18
10. Installation Guide 18
Prerequisites 18
Build and Stage 18
Initial Application Setup 18
11. Deployment Guide 19

Production Topology 19
Service Requirements 19
Deployment Sequence 19
Verification Checklist 19
Rollback and Continuity 20
12. User Workflows 20
First Installation — Admin 20
Daily Mixing — Operator 20
Prepare and Recall a Show 20
User Administration — Admin 20
Read-Only Monitoring — Viewer 21
MIDI Interface Reconnection 21
13. Operational Procedures 21
Daily Start-of-Service 21
Backup Procedure 21
Restore Procedure 21
Account Lifecycle 22
Change Management 22
14. Monitoring and Logging 22
Logging Strategy 22
Monitoring Matrix 22
Important Log Events 22
15. Troubleshooting Guide 23
Escalation Package 24
16. FAQ 24
Can multiple people use the system simultaneously? 24
Does an application scene equal a console scene? 24
Why did a captured scene omit a setting? 24
Can the server run on Windows or macOS? 24
Is HTTPS required? 25
What must be backed up? 25
Why are app changes rejected when MIDI output is unavailable? 25

How quickly does the system recover after USB reconnection? 25
Are all processing graphs exact? 25
17. Appendix and Glossary 25
Appendix A — Mixer Module Map 25
Appendix B — Parameter Rules 25
Appendix D — Consulting Recommendations 26
Glossary 26

1. Executive Summary

DDX3216 Remote is a browser-based control platform for the Behringer DDX3216 32-channel digital mixing console. It bridges the console's MIDI-only interface to modern phones, tablets, and computers through a compact Linux server, typically a Raspberry Pi. The server hosts the user interface, REST API, live WebSocket synchronization, persistence, user management, and ALSA-based MIDI routing.

The platform enables operators to mix from anywhere on the local network, coordinate multiple authenticated users, apply descriptive fader names and colors, visualize signal processing, and prepare repeatable application-side scenes. Two-way synchronization keeps browsers aligned with console changes when both MIDI directions are connected.

Primary value: improved mobility, usability, repeatability, and collaborative control without requiring a native client installation.

Key constraints: the application controls one desk per installation; MIDI is Linux-only; app scenes are independent from console scenes; and snapshots include only values known to the server.

Recommendations

- Deploy behind nginx and enable TLS whenever the network is not fully trusted.
- Preserve and back up the complete App_Data directory before every upgrade.
- Validate assumptions concerning FX return mute, route-to-bus bitmasks, and pre/post compressor behavior on the physical console.
- Restrict public diagnostics and Swagger in production.

2. Business Overview

Business Objectives

Objective	Business outcome
Remote room control	Operators mix from audience or stage positions using a touch-friendly browser interface.
Safe multi-user operation	Admin, Operator, and Viewer roles combine controlled access with live synchronization.
Simplified operation	Custom labels, colors, processing graphs, and organized effects reduce cognitive load.
Repeatable events	Scenes store labels and optional known-value snapshots for preparation and recall.

Low-cost reliability	A Raspberry Pi host provides automatic service restart, MIDI reconnection, and persistent state.

Stakeholders

- **Business owners and project managers:** service value, constraints, risk, and continuity.
- **Administrators:** identity, configuration, deployment, backup, and security.
- **Operators:** live mixing, processing, effects, scenes, and MIDI status.
- **Support teams:** health verification, logs, diagnostics, and incident restoration.
- **Developers:** React client, ASP.NET Core services, API contracts, database, and MIDI protocol.

Scope and Limitations

The solution supports one active application scene, one MIDI input, and one MIDI output per installation. The server exposes 20 documented endpoints, including authenticated application APIs, the WebSocket endpoint, the health endpoint, and Swagger UI.

3. System Architecture

Logical Architecture

Client tier	Browser-based React single-page application on phone, tablet, or computer
Edge tier	nginx on port 80, forwarding HTTP and WebSocket traffic to the local application
Application tier	ASP.NET Core 10 server: REST controllers, WebSocket hub, change pipeline, scenes, users, and MIDI router
Data tier	SQLite through Entity Framework Core; JSON MIDI settings; Data Protection keys
Device tier	ALSA sequencer → USB MIDI interface → MIDI DIN SysEx → Behringer DDX3216

Architecture Diagram

Browser clients React · REST · WebSocket	⇄	**nginx** Reverse proxy	⇄	ASP.NET Core 10 API · hub · services
		↙	↓	↘

SQLite Users · scenes · values		**App_Data** Settings · cookie keys		**ALSA → USB MIDI → DDX3216**

Figure 1. High-level deployment and integration topology.

Primary Components

Component	Responsibility
Web client	Authentication, live control, scenes, user management, and responsive presentation.
REST API	Authentication, users, scenes, MIDI setup, state queries, and diagnostics.
ClientHub	WebSocket connections, snapshots, status, scene updates, queues, and heartbeat handling.
ChangeBus / ChangeDispatcher	Single ordered pipeline for browser, console, and scene-recall changes.
MidiOutputQueue	Latest-value coalescing, batching, and output pacing.
MidiRouterService	ALSA port discovery, selection, hot-plug reconnection, and SysEx transport.
SceneService / UserService	Scene lifecycle and identity/role lifecycle.
Persistence services	SQLite schema, live-state recovery, and periodic saving.

Change Data Flow

1. A browser, console, or scene recall produces one or more parameter changes.
2. Ingress validates module, parameter, value, role, and MIDI availability.
3. ChangeBus preserves arrival order.
4. ChangeDispatcher updates live state, suppresses echoes, and routes changes.
5. MidiOutputQueue coalesces values, packs up to 23 changes per SysEx message, and paces output.
6. ClientHub broadcasts the authoritative updates to connected browsers.
7. LiveStatePersistence saves changed values every two seconds and at shutdown.

For operational symptoms affecting this flow, see Section 15, Troubleshooting Guide.

4. Functional Overview

Core Capabilities

- **Mixer view:** input channels 1–32, buses 1–16, auxiliary and effects masters/returns, and a pinned Main strip.
- **Processing view:** four-band EQ, high-pass filtering where supported, compressor, gate, delay, routing, sends, and live response graphs.
- **Effects view:** four slots and 27 effect choices including Bypass, grouped by category with effect-specific parameters.
- **Two-way synchronization:** console and browser changes propagate to all active clients.
- **Scenes:** one installation-wide active scene with labels, colors, and optional known-value snapshot.
- **Identity:** first-run administrator creation, local users, role-based access, and session invalidation.
- **MIDI administration:** port selection, channel configuration, status display, refresh, persistence, and hot-plug recovery.

Role Matrix

Capability	Admin	Operator	Viewer
View system state	Yes	Yes	Yes
Change mixer parameters	Yes	Yes	No
Manage MIDI and scenes	Yes	Yes	No
Change own password	Yes	Yes	Yes
Manage users and roles	Yes	No	No

Business Rules

- At least one Admin must always exist; users cannot delete their own account.
- User names are case-insensitively unique; scene names are also case-insensitively unique.
- The last scene cannot be deleted, and exactly one active scene exists per installation.
- Viewer writes are rejected by both client behavior and server enforcement.
- Browser changes are refused when no MIDI output is connected.
- Snapshot capture stores only values known to the server.
- Console scene changes must be matched manually with the corresponding application scene.

5. Technical Architecture

Technology Stack

Layer	Technology	Version
Web UI	React	19.2
State and data	Redux Toolkit, RTK Query, react-redux	2.11 / 9.2
Styling	Bootstrap and custom CSS	5.3
Build	Vite	7.3
API client generation	@rtk-query/codegen-openapi	2.2
Server	ASP.NET Core / C# / Kestrel	.NET 10
Database	SQLite through Entity Framework Core	EF Core 10.0.12
MIDI	ALSA sequencer through libasound.so.2	System library
API description	Swashbuckle / OpenAPI 3.0.4	10.3.0

Runtime and Build Dependencies

- Linux with ALSA, libasound2, and access to /dev/snd/seq; the service account belongs to the audio group.
- A SysEx-capable USB MIDI interface connected to the console MIDI IN and OUT.
- The server is published as a self-contained single file, so the .NET runtime is not required on the host.
- Key packages: Microsoft.EntityFrameworkCore.Sqlite 10.0.12, Microsoft.EntityFrameworkCore.Design 10.0.12, Swashbuckle.AspNetCore 10.3.0, Microsoft.AspNetCore.SpaProxy 10.0.12, and Microsoft.VisualStudio.Azure.Containers.Tools.Targets 1.23.0.

Frontend Design

The React client uses function components and hooks. Redux maintains WebSocket state and parameter values; RTK Query manages REST-backed authentication, users, scenes, and MIDI status. Parameter definitions drive control type, range, labels, and display formatting. The central control hook applies local drag overrides, limits transmission to one update per 30 milliseconds, always sends the final value, and prevents Viewer writes.

Backend Design

The backend separates presentation, application, domain, and infrastructure concerns. Singleton and hosted services coordinate state, changes, WebSockets, MIDI, scenes, users, settings, and persistence. Short-lived database contexts are created through an

EF Core context factory. Events remove circular dependencies between MIDI status, active scenes, session invalidation, and the client hub.

SysEx Protocol

Parameter-change messages use the structure **F0 00 20 32 <ic> 0B 20 <nn> [module param valueHi valueLo] × nn F7**. Values are 14-bit quantities carried as two 7-bit bytes. A message contains between one and 23 changes.

Best practice: Keep output pacing near the documented 3,000 bytes per second unless physical-console testing establishes a safe alternative.

.NET 10 Upgrade Validation

- The server builds without warnings on the .NET 10 SDK 10.0.401.
- Fresh-database migrations apply successfully and the health endpoint responds.
- Swagger UI loads and lists all 20 endpoints.
- Twenty-nine API tests covering login, roles, scenes, labels, sessions, and rate limiting pass.
- The generated API client matches the repository version.
- Raspberry Pi execution has not yet been validated.

6. Database Design

The application uses SQLite through EF Core 10. The default database is App_Data/ddx3216.db. Migrations are applied automatically at startup, and write-ahead logging is enabled.

Entity Summary

Entity	Key	Purpose	Important constraints
Users	Id	Local accounts, roles, password hashes, security stamps	UserName unique with NOCASE; Role required
Scenes	Id	Named setup and snapshot metadata	Name unique with NOCASE
SceneLabels	SceneId + Module	Per-scene module name and color	Cascade delete with scene
SceneParameters	SceneId + Module + Param	Per-scene parameter snapshot	Cascade delete with scene

LiveParameters	Module + Param	Last-known mixer values	Standalone persistence
AppState	Id = 1	Installation-wide active scene	ActiveSceneId set null on scene deletion

Relationship Diagram

Scenes	1 → many	**SceneLabels**
Scenes	1 → many	**SceneParameters**
Scenes	1 ← optional	**AppState.ActiveSceneId**
Users	informational only	**Scenes.UpdatedByUserId** (no FK)
LiveParameters	standalone	Loaded on startup; saved every 2 seconds

Data Protection and Backup

Back up the database together with **midi-settings.json** and the **keys** directory. Losing the keys signs out every user; losing the database removes users, scenes, labels, and persisted values. Stop the service or use a consistent SQLite-aware backup method before copying live files.

7. Authentication and Security

Authentication Model

The system uses local accounts and an ASP.NET Core authentication cookie named **ddx3216.auth**. There are no bearer tokens, JWTs, OAuth providers, or directory integrations. The cookie is HttpOnly, SameSite=Strict, persistent, and uses a 30-day sliding expiration. It is Secure when the request uses HTTPS.

Session and Authorization Flow

1. The client requests authentication state.
2. If no users exist, first-run setup creates the first Admin and signs in.
3. Login verifies a PBKDF2 password hash and issues the cookie.
4. Each request reloads the user and validates the security stamp.
5. Role or password changes and user deletion rotate the stamp and close affected WebSockets.

6. The WebSocket upgrade requires the same authenticated cookie and an allowed Origin.

Security Controls and Considerations

Area	Current control	Recommendation
Password storage	PBKDF2 through PasswordHasher; hash upgrades on login	Use long unique passwords and periodic access review.
Brute force	10 setup/login requests per minute per client IP	Monitor failed-login logs; consider persistent rate limiting.
CSRF	SameSite=Strict cookie and JSON-only API	Retain Strict mode and Origin validation.
Transport	Plain HTTP on LAN by default	**Enable TLS at nginx on untrusted or shared networks.**
Cookie keys	Stored unencrypted under App_Data/keys	**Restrict file permissions and include keys in protected backups.**
Diagnostics	Health, route list, debug endpoint, and Swagger are anonymous	**Restrict by authentication, firewall, or reverse-proxy rules.**
CORS	REST allows any origin; cookies remain Strict	Replace broad CORS with explicit trusted origins.
Auditability	No parameter-level audit log	Add structured security and configuration audit events.

See Section 9 for security-relevant configuration and Section 14 for security monitoring.

8. API Reference

Conventions

- Base URL: **http://<host>/**.
- Payloads are JSON in camelCase, except parameter changes use Module, Param, and Value.
- Authentication uses the ddx3216.auth cookie.
- OpenAPI is available at /swagger/v1/swagger.json and Swagger UI at /swagger.

HTTP Status Codes

Status	Meaning

400	Invalid input
401	Not signed in, expired session, or revoked session
403	Authenticated but insufficient role
404	Resource not found
409	Business-rule conflict
429	Login/setup rate limit exceeded

Endpoint Catalog

Method and route	**Access**	**Purpose**
GET /api/auth/state	Anonymous	Return setup requirement and current user.
POST /api/auth/setup	Anonymous; first run	Create first Admin and issue cookie.
POST /api/auth/login	Anonymous	Authenticate and issue cookie.
POST /api/auth/logout	Anonymous	Clear cookie.
POST /api/auth/password	Authenticated	Change own password.
GET /api/users	Admin	List users.
POST /api/users	Admin	Create user.
PUT /api/users/{id}	Admin	Change role and/or reset password.
DELETE /api/users/{id}	Admin	Delete user subject to Admin rules.
GET /api/scenes	Authenticated	List scene summaries.
GET /api/scenes/active	Authenticated	Return active scene and labels.
POST /api/scenes	Admin/Operator	Create a scene with optional capture and copied labels.
PUT /api/scenes/{id}	Admin/Operator	Rename a scene.
DELETE /api/scenes/{id}	Admin/Operator	Delete a non-final scene.

POST /api/scenes/{id}/capture	Admin/Operator	Replace snapshot with current known values.
POST /api/scenes/{id}/activate	Admin/Operator	Activate labels only or labels plus mixer values.
PUT /api/scenes/{id}/labels/{module}	Admin/Operator	Set or remove a module label and color.
GET /api/Midi/devices	Authenticated	Return ports, selections, channel, and connection status.
GET /api/Midi/state	Authenticated	Return all known parameter values.
GET /api/Midi/status/{module}/{param}/{default}	Authenticated	Legacy single-value query.
POST /api/Midi/out/select/{port}	Admin/Operator	Select and connect MIDI output.
POST /api/Midi/in/select/{port}	Admin/Operator	Select and connect MIDI input.
POST /api/Midi/channel/{channel}	Admin/Operator	Set channel 1–16.
POST /api/Midi/devices/refresh	Admin/Operator	Re-enumerate and reconnect ports.

Operational Endpoints

Route	**Access**	**Purpose**
GET /health	Anonymous	Status, clients, MIDI connections, queue, channel, and active scene.
GET /swagger	Anonymous	Swagger UI.

WebSocket Contract

The endpoint **/ws/midi** requires authentication and an allowed Origin. Clients send change batches or ping messages. The server sends snapshot, status, scene, changes, error, and pong messages.

Message type	Direction	Purpose
changes	Both	Submit or distribute parameter changes.
ping / pong	Both	Heartbeat and idle detection.
snapshot	Server → client	All known values on connect or resynchronization.
status	Server → client	MIDI device, selection, connection, and channel status.
scene	Server → client	Current active scene and labels.
error	Server → client	Invalid message, read-only write, or unavailable MIDI output.

9. Configuration Guide

Settings may be supplied through appsettings.json or environment variables. Environment-variable names replace each colon with a double underscore.

Application Settings

Setting	Default	Purpose	Operational guidance
DataDirectory	<content root>/App_Data	Database, keys, and MIDI settings	Place on protected, backed-up storage.
Database:Path	<DataDirectory>/ddx3216.db	SQLite database	Do not overwrite during upgrades.
Midi:SettingsPath	<DataDirectory>/midi-settings.json	Saved port and channel selection	Preserve with the database.
Midi:FilterIncomingChannel	true	Filter incoming SysEx	Disable only temporarily for diagnosis.
Midi:MaxBytesPerSecond	3000	Output pacing; minimum 500	Keep near the MIDI DIN capacity.

Midi:PollSeconds	10	Fallback port check interval	Lower only with measured need.
WebSocket:CheckOrigin	true	Origin enforcement	**Do not disable in production.**
WebSocket:AllowedOrigins	[]	Additional trusted origins	Use exact scheme and host entries.
WebSocket:MaxMessageBytes	65536	Incoming message limit	Retain unless a validated workload requires change.
WebSocket:QueueCapacity	512	Per-client send queue	Monitor resynchronization warnings before tuning.
WebSocket:SendTimeoutSeconds	5	Blocked-send disconnect threshold	Balance slow-client tolerance against resource use.
WebSocket:IdleTimeoutSeconds	30	Silent-client disconnect threshold	Keep above the 10-second ping period.
Logging:LogLevel:*	Information; framework Warning	Log verbosity	Increase temporarily during incidents.

Environment

Variable	**Purpose**
ASPNETCORE_URLS	Production listen address; recommended http://127.0.0.1:5000 behind nginx.
ASPNETCORE_ENVIRONMENT	Production or Development behavior.
Midi__MaxBytesPerSecond	Example override for a nested application setting.
DEV_SERVER_PORT	Vite development port; default 56578.

Fixed Values

Cookie lifetime, login rate limit, password minimum, name limits, live-save interval, echo window, and client throttle are fixed in code. Changes require source modification and regression testing.

10. Installation Guide

Prerequisites

Item	Requirement
Host	Raspberry Pi or compatible 64-bit ARM Linux host for the linux-arm64 build.
Packages	libasound2 and nginx; container images based on Ubuntu 24.04 use libasound2t64.
MIDI	SysEx-capable USB MIDI interface connected to console MIDI IN and OUT.
Permissions	Service account in the audio group with access to /dev/snd/seq.
Build workstation	.NET 10 SDK and Node.js 18 or later.
Network	Host and user devices on the same reachable LAN.

Build and Stage

1. Publish the server in Release mode using the linux-arm64, self-contained, single-file profile targeting net10.0.
2. Transfer the published output to a new staging directory on the host.
3. Stop any existing ddx3216 service.
4. Back up the complete App_Data directory.
5. Copy staged files into the application directory without deleting App_Data.
6. Restore executable permission on DDX3216MidiControl.Server.
7. Add the service account to the audio group.

Initial Application Setup

1. Open the site in a browser.
2. Create the first Admin account.
3. Open MIDI settings, select the physical USB interface for both directions, and match the console MIDI channel.
4. Confirm the MIDI indicator is green.
5. Create Operator and Viewer accounts as required.

Security consideration: Complete first-run Admin setup immediately after deployment so the anonymous setup endpoint cannot be used by an unintended user.

11. Deployment Guide

Production Topology

Run the application as a dedicated systemd service listening on loopback at port 5000. Expose the service through nginx, forwarding HTTP, Host, X-Forwarded-For, X-Forwarded-Proto, Upgrade, and Connection headers. Set a long proxy read timeout for WebSocket sessions.

Service Requirements

- Run under a non-root service account.
- Set the working directory to the deployed application directory.
- Grant the audio supplementary group.
- Set ASPNETCORE_URLS to http://127.0.0.1:5000.
- Set ASPNETCORE_ENVIRONMENT to Production.
- Configure automatic restart with a short delay.

Deployment Sequence

1. Validate nginx configuration.
2. Stop the application service.
3. Create a timestamped App_Data backup.
4. Install the new binary and web assets.
5. Reload systemd configuration when the unit changes.
6. Start or enable the service.
7. Reload nginx.
8. Run the verification checklist below.

Verification Checklist

Check	**Expected result**
Service status	active (running)
Startup log	Database path shown; ALSA sequencer ready or a clear MIDI-disabled reason
Local health	/health returns JSON with status ok
Reverse proxy	/api/auth/state returns JSON rather than HTML

ALSA	Application clients and USB MIDI interface are listed
Browser UI	Login or first-run setup screen appears
App → console	Moving a browser fader affects the console
Console → app	Moving a console fader updates the browser

Rollback and Continuity

Retain the previous application package until verification succeeds. To roll back, stop the service, restore the previous binaries, preserve or restore the compatible App_Data backup, then start and reverify. Database migrations are automatic; therefore test rollback compatibility before production upgrades.

12. User Workflows

First Installation — Admin

1. Deploy and verify the server as described in Sections 10 and 11.
2. Create the initial Admin account.
3. Select MIDI input and output and configure the channel.
4. Create least-privilege user accounts.

Daily Mixing — Operator

1. Sign in and confirm the MIDI indicator is green.
2. Select the desired channel, bus, auxiliary/effects, processing, or effects view.
3. Adjust levels, mute, pan, dynamics, EQ, routing, and effects as required.
4. Confirm other clients and the console follow live changes.

Prepare and Recall a Show

1. Configure the console and ensure MIDI IN is connected so the server knows current values.
2. Create or select a scene and save current mixer values.
3. Before recall, confirm MIDI output is connected and the correct scene is selected.
4. Use **Load names + mixer** for full recall or **Load names** when only labels should change.

User Administration — Admin

1. Open Account → Users.
2. Add users with the minimum required role.
3. Change roles or reset passwords when duties change.

4. Delete unused accounts, while retaining at least one Admin.

Read-Only Monitoring — Viewer

Viewers can observe mixer, processing, effects, scenes, and MIDI status. Controls remain read-only, while live changes continue to display.

MIDI Interface Reconnection

When the interface is unplugged, browser writes are refused and the status turns amber. Reconnect the same interface; selection by name allows automatic recovery, normally within about one second.

13. Operational Procedures

Daily Start-of-Service

1. Confirm the application service is active.
2. Review recent warnings and errors.
3. Check /health for status ok, MIDI input/output connected, queue near zero, and expected active scene.
4. Open the UI and confirm authenticated access.
5. Test one non-disruptive app-to-console and console-to-app change.

Backup Procedure

1. Choose a quiet operational window.
2. Stop the service for a simple consistent copy, or use an approved SQLite-consistent method.
3. Back up the entire App_Data directory, including database, WAL-related files if present, MIDI settings, and Data Protection keys.
4. Store the backup with restricted access and test restoration periodically.
5. Restart the service and verify health.

Restore Procedure

1. Stop the service.
2. Preserve the current App_Data as a forensic or fallback copy.
3. Restore the approved backup with correct ownership and permissions.
4. Start the service and verify users, scenes, MIDI settings, and bidirectional control.

Account Lifecycle

Provision named accounts, assign least privilege, review accounts regularly, reset credentials on suspected compromise, and remove accounts promptly. Role or password changes invalidate existing sessions.

Change Management

Record application version, commit, configuration changes, deployment date, backup identifier, verification outcome, and rollback decision. Validate console-specific assumptions before promoting protocol-related changes.

14. Monitoring and Logging

Logging Strategy

The server uses Microsoft.Extensions.Logging. Under systemd, console output is captured by the journal. Application events default to Information; ASP.NET Core and EF Core default to Warning.

Monitoring Matrix

Signal	Source	Expected	Response
Service availability	systemd	active	Review journal and restart cause.
Application health	GET /health	status ok	Escalate if repeated failures occur.
MIDI output	/health	connected during operation	Check USB, ALSA, selection, and permissions.
MIDI queue	/health	Returns to zero within seconds	Investigate connection or pacing if sustained.
Client health	Journal	No repeated timeout or resync messages	Check network and proxy WebSocket settings.
Security events	Journal	No unusual failed logins or rejected origins	Review source and account status.
Storage	App_Data usage	Stable and writable	Check free space, permissions, and WAL behavior.

Important Log Events

- Startup database path, restored-value count, and ALSA readiness.

- MIDI selection, connection, disconnection, send failure, and input overrun.
- Client connection count, queue overflow, timeout, and idle disconnect.
- Failed login and rejected WebSocket Origin.
- Scene creation, deletion, activation, and first Admin creation.
- Dispatch, persistence, and MIDI-batch failures.

Best practice: Add an external uptime check for /health and alert when MIDI output is disconnected during scheduled operations. Built-in metrics and alerting are not provided.

15. Troubleshooting Guide

Symptom	Likely cause	Resolution
Service status 203/EXEC	Executable permission missing	Restore execute permission and restart.
“Text file busy” during copy	Old process still running	Stop the service before replacement.
UI cannot reach server or /health returns HTML	Application down, stale version, or proxy misroute	Verify local port 5000, deployed binary, and nginx upstream.
Page loads but MIDI status is Offline	WebSocket proxy or Origin failure	Forward Upgrade, Connection, and Host; inspect rejected-Origin logs.
MIDI disabled	No ALSA access or unsupported OS	Use Linux; grant audio group and /dev/snd/seq access.
Set up MIDI	No output selected	Select the physical USB interface.
Loopback only	Midi Through selected	Select the hardware interface instead.
Output unplugged	USB interface disconnected	Reconnect and wait for automatic recovery.
App changes do not reach console	Wrong channel, reversed cables, or SysEx disabled	Match channel; connect output to console input; check console settings.
Console changes do not reach app	Input not selected or channel filtered	Select input and match channel; temporarily disable filtering only for diagnosis.
Too many login attempts	10-per-minute limit reached	Wait one minute and retry.

Everyone signed out after update	Data Protection keys replaced	Preserve App_Data; users must sign in again if keys are lost.
Users or scenes missing	Database removed	Stop service and restore App_Data backup.
Port 80 conflict	Kestrel and nginx both bind port 80	Bind Kestrel to loopback port 5000.
Read-only account	User has Viewer role	Admin assigns Operator or Admin only if appropriate.
Scene recall returns 409	No MIDI output open	Connect output or load names only.
Scene contains names only	No known values at capture	Establish MIDI input or move controls, then capture again.
No MIDI on Windows development	ALSA is Linux-only	Expected; test MIDI on Linux.

Escalation Package

Provide the application version and commit, host OS and architecture, deployment topology, recent relevant logs, /health output with sensitive details reviewed, MIDI interface model, channel, reproduction steps, expected behavior, actual behavior, and whether both MIDI directions are affected.

16. FAQ

Can multiple people use the system simultaneously?

Yes. Connected browsers receive live updates. Role controls determine who may change the mixer or administer users.

Does an application scene equal a console scene?

No. They are independent. The console does not report its scene through the implemented live SysEx protocol.

Why did a captured scene omit a setting?

Snapshots contain only values known to the server. Connect MIDI IN or exercise relevant controls before capture.

Can the server run on Windows or macOS?

The web and database layers can run without MIDI, but the implemented MIDI integration requires Linux ALSA.

Is HTTPS required?

It is strongly recommended on shared or untrusted networks. Terminate TLS at nginx so authentication cookies become Secure.

What must be backed up?

The entire App_Data directory: database, MIDI settings, and Data Protection keys.

Why are app changes rejected when MIDI output is unavailable?

The server avoids presenting an unconfirmed state that was never sent to the console.

How quickly does the system recover after USB reconnection?

The saved port name and hot-plug monitoring normally restore the connection automatically.

Are all processing graphs exact?

No. EQ and compressor visualizations are engineering approximations rather than exact console DSP models.

17. Appendix and Glossary

Appendix A — Mixer Module Map

Module range	Meaning
0–31	Input channels 1–32
32–47	Buses 1–16
48–51	Auxiliary masters 1–4
52–55	Effects sends/masters 1–4
56–63	Effects returns 1–4 as left/right pairs
64	Main stereo output

Appendix B — Parameter Rules

- Valid module: 0–127; valid parameter: 0–127; valid raw value: 0–16383.
- Volume raw range 0–1472; dB = raw ÷ 16 − 80; 0 dB = 1280.
- Pan/balance raw range 0–60; center = 30.
- Effect selection uses parameter 90 on modules 52–55; effect parameters use 91–98.
- Outgoing updates coalesce by parameter and batch up to 23 changes.

- Incoming values received within two seconds of an application send may be suppressed as echoes.

Appendix D — Consulting Recommendations

1. **Security:** add TLS, restrict diagnostics and Swagger, narrow CORS, protect App_Data, and document account reviews.
2. **Reliability:** automate versioned backups and restore tests; add external health and MIDI-link alerting.
3. **Quality:** introduce automated unit, integration, migration, WebSocket, and protocol tests.
4. **Observability:** add structured logs, correlation identifiers, metrics, and an auditable configuration-change trail.
5. **Maintainability:** remove unused client state and legacy endpoints after compatibility review.
6. **Validation:** test inferred console behaviors and maintain an approved hardware/firmware compatibility matrix.

Glossary

Term	Definition
DDX3216	Behringer 32-channel digital mixing console controlled by the application.
MIDI	Serial musical-device protocol used to communicate with the console.
SysEx	Manufacturer-specific MIDI message framed by F0 and F7.
ALSA	Linux audio architecture whose sequencer routes MIDI applications and devices.
Module	Console unit to which a parameter belongs, such as a channel, bus, return, or Main.
Param	Parameter identifier within a module.
Raw value	Fourteen-bit SysEx value converted into user-facing units.
Scene	Application-stored labels and optional known-value snapshot.
Capture	Save currently known values into a scene.
Recall	Send saved scene values to the console.
Snapshot	Full known-value set sent to a browser or stored in a scene.
Security stamp	Per-user random value whose rotation invalidates authentication cookies.

ChangeBus	Single ordered queue for all parameter changes.
Coalescing	Retaining only the latest pending value for each parameter.
Pacing	Limiting output throughput to MIDI transport capacity.
Echo suppression	Ignoring returned MIDI values that duplicate a recent application send.
Hot-plug	Detecting and recovering a device connection while running.
WebSocket	Persistent two-way browser/server connection for live state.
WAL	SQLite write-ahead logging mode.
Data Protection keys	ASP.NET Core keys used to sign and encrypt authentication cookies.

End of manual.